



Worksheet 1

Student Name: HARSH

Branch: MCA

Semester: 3

Subject Name: BACKEND TECHNOLOGY LAB

UID: 24MCA20045

Section/Group: 1-B

Date of Performance: 23/07/2025

Subject Code: 24CAP-709

Aim/Overview of the practical:

Q. Create employee salary program using NodeJS.

1. **Task to be done:** Develop a command-line application that allows for the management of employee records, including the calculation and tracking of salaries, using Node.js and JSON files for data storage.

2. Algorithm/Flowchart:

Step 1: Start App

Read employees.json. If not present, create it as [].

Step 2: Show Main Menu

Loop: show menu options until user exits.

Step 3: Handle Menu Options

Add Employee:

1. Prompt: name, age, department, hourlyRate, hoursWorked.
2. Generate unique ID.
3. Calculate salary = hourlyRate * hoursWorked.
4. Add to JSON array and save.

View All Employees:

1. Read from employees.json.
2. Display in tabular form using console.table().

Search Employee:

1. Ask for name or ID.
2. Match from array.
3. Display if found.

Update Employee:

1. Ask for ID.
2. Show current details.
3. Ask what to update (e.g., hours worked).
4. Recalculate salary and save.

Delete Employee:

1. Ask for ID.
2. Confirm deletion.
3. Remove from array and save.

Salary Report:

1. Calculate salaries again if needed.
2. Summarize total payroll, average salary, etc.
3. Display data.

Step 4: Exit App

- Save data to JSON again just in case.
- Exit gracefully

3. Code for experiment/practical:

Input:

App.js

```
const fs = require('fs');
const readline = require('readline');
const path = require('path');
const dataFilePath = path.join(__dirname, 'data', 'employees.json');
```

```
// Function to read employee data from the JSON file
```

```
function readEmployeeData() {
  try {
    const data = fs.readFileSync(dataFilePath, 'utf8');
    return JSON.parse(data);
  } catch (err) {
    console.error('Error reading employee data:', err);
    return [];
  }
}
```

```
// Function to write employee data to the JSON file
```

```
function writeEmployeeData(data) {
  try {
```

```
    fs.writeFileSync(dataFilePath, JSON.stringify(data, null, 2));
  } catch (err) {
    console.error('Error writing employee data:', err);
  }
}
```

```
// Function to calculate salary
```

```
function calculateSalary(hourlyRate, hoursWorked) {
  return hourlyRate * hoursWorked;
}
```

```
// Function to display the main menu
```

```
function displayMenu() {
  console.log('\nEmployee Salary Management System');
  console.log('1. Add Employee');
  console.log('2. View Employees');
  console.log('3. Update Employee');
  console.log('4. Delete Employee');
  console.log('5. Exit');
  rl.question('Enter your choice: ', handleMenu);
}
```

```
// Function to handle menu choices
```

```
function handleMenu(choice) {
  switch (choice) {
    case '1':
      addEmployee();
      break;
    case '2':
      viewEmployees();
      break;
    case '3':
      updateEmployee();
      break;
    case '4':
      deleteEmployee();
      break;
    case '5':
      rl.close();
      break;
    default:
      console.log('Invalid choice. Please try again. ');
      displayMenu();
  }
}
```

}

// Function to add a new employee

```
function addEmployee() {
  rl.question('Enter employee name: ', (name) => {
    rl.question('Enter position: ', (position) => {
      rl.question('Enter hourly rate: ', (hourlyRate) => {
        rl.question('Enter hours worked: ', (hoursWorked) => {
          const employees = readEmployeeData();
          const id = employees.length ? employees[employees.length - 1].id + 1 : 1;
          const salary = calculateSalary(parseFloat(hourlyRate), parseFloat(hoursWorked));
          employees.push({ id, name, position, hourlyRate: parseFloat(hourlyRate), hoursWorked:
parseFloat(hoursWorked), salary });
          writeEmployeeData(employees);
          console.log('Employee added successfully.');
```

displayMenu();

});

});

});

});

}

// Function to view all employees

```
function viewEmployees() {
  const employees = readEmployeeData();
  if (employees.length === 0) {
    console.log('No employee records found.');
```

} else {

console.log("\nEmployee Records:");

employees.forEach((emp) => {

```
  console.log(' ID: ${emp.id}, Name: ${emp.name}, Position: ${emp.position}, Hourly Rate:
${emp.hourlyRate}, Hours Worked: ${emp.hoursWorked}, Salary: ${emp.salary}');
```

});

}

displayMenu();

}

// Function to update an existing employee

```
function updateEmployee() {
  rl.question('Enter employee ID to update: ', (id) => {
    const employees = readEmployeeData();
    const employee = employees.find((emp) => emp.id === parseInt(id));
    if (!employee) {
      console.log('Employee not found.');
```

displayMenu();

```
} else {
  rl.question(`Enter new name (${employee.name}): `, (name) => {
    rl.question(`Enter new position (${employee.position}): `, (position) => {
      rl.question(`Enter new hourly rate (${employee.hourlyRate}): `, (hourlyRate) => {
        rl.question(`Enter new hours worked (${employee.hoursWorked}): `, (hoursWorked) => {
          employee.name = name || employee.name;
          employee.position = position || employee.position;
          employee.hourlyRate = hourlyRate ? parseFloat(hourlyRate) : employee.hourlyRate;
          employee.hoursWorked = hoursWorked ? parseFloat(hoursWorked) : employee.hoursWorked;
          employee.salary = calculateSalary(employee.hourlyRate, employee.hoursWorked);
          writeEmployeeData(employee);
          console.log('Employee updated successfully.');
```

```
          displayMenu();
        });
      });
    });
  });
}
});
}

// Function to delete an employee

function deleteEmployee() {
  rl.question('Enter employee ID to delete: ', (id) => {
    let employees = readEmployeeData();
    const index = employees.findIndex((emp) => emp.id === parseInt(id));
    if (index === -1) {
      console.log('Employee not found.');
```

```
    } else {
      employees.splice(index, 1);
      writeEmployeeData(employees);
      console.log('Employee deleted successfully.');
```

```
    }
    displayMenu();
  });
}

// Create readline interface

const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout,
});

// Start the application
displayMenu();
```

4. Result/Output/Writing Summary:

1. Add Employee

```
Employee Salary Management System
1. Add Employee
2. View Employees
3. Update Employee
4. Delete Employee
5. Exit
Enter your choice: 1
Enter employee name: Harsh
Enter position: HR
Enter hourly rate: 3000
Enter hours worked: 8
Employee added successfully.

Employee Salary Management System
1. Add Employee
2. View Employees
3. Update Employee
4. Delete Employee
5. Exit
Enter your choice: 1
Enter employee name: Varn
Enter position: Data scientist
Enter hourly rate: 4000
Enter hours worked: 12000
Employee added successfully.
```

2. View Employees

```
Employee Salary Management System
1. Add Employee
2. View Employees
3. Update Employee
4. Delete Employee
5. Exit
Enter your choice: 2

Employee Records:
ID: 1, Name: Harsh, Position: HR, Hourly Rate: 3000, Hours Worked: 8, Salary: 24000
ID: 2, Name: Varn, Position: Data scientist, Hourly Rate: 4000, Hours Worked: 12000, Salary: 48000000
```

3. Update Employees

```
Employee Salary Management System
1. Add Employee
2. View Employees
3. Update Employee
4. Delete Employee
5. Exit
Enter your choice: 3
Enter employee ID to update: 2
Enter new name (Varn): Rahul
Enter new position (Data scientist): Web developer
Enter new hourly rate (4000): 5000
Enter new hours worked (12000): 9000
Employee updated successfully.
```

4. Delete Employee

Employee Salary Management System

1. Add Employee
2. View Employees
3. Update Employee
4. Delete Employee
5. Exit

Enter your choice: 4

Enter employee ID to delete: 2

Employee deleted successfully.

Employee Salary Management System

1. Add Employee
2. View Employees
3. Update Employee
4. Delete Employee
5. Exit

Enter your choice: 2

Employee Records:

ID: 1, Name: Harsh, Position: HR, Hourly Rate: 3000, Hours Worked: 8, Salary: 24000

Learning outcomes (What I have learnt):

1. Learned how to read, write, update, and delete JSON files.
2. Applied CRUD operations without a database — using file-based persistence.
3. Gained experience working with CLI prompts using readline or inquirer.
4. Handled invalid input, file read/write errors, missing records, etc.
5. Practiced searching, filtering, and updating data inside JSON arrays.

Evaluation Grid:

Sr. No.	Parameters	Marks Obtained	Maximum Marks
1.	Demonstration and Performance		12
2.	Worksheet		8
3.	Via		10